

Week10

Task1:

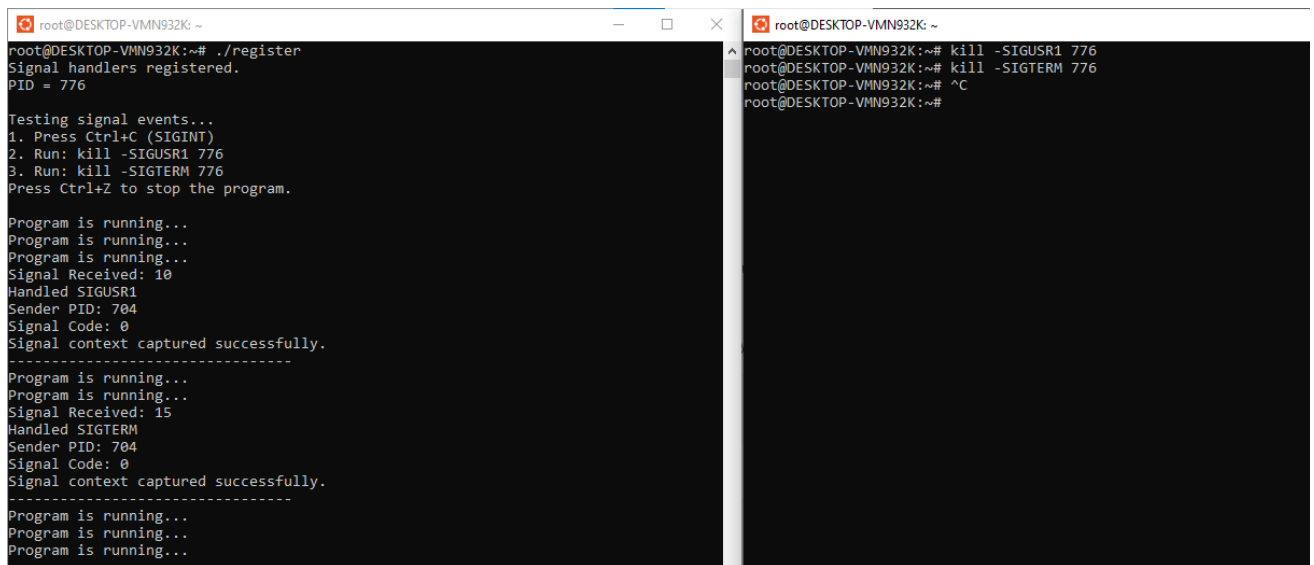
To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>
void signalHandler(int signo, siginfo_t *info, void *context)
{
    printf("Signal Received: %d\n", signo);

    switch (signo)
    {
        case SIGINT:
            printf("Handled SIGINT (Ctrl+C)\n");
            break;
        case SIGTERM:
            printf("Handled SIGTERM\n");
            break;
        case SIGUSR1:
            printf("Handled SIGUSR1\n");
            break;
        default:
            printf("Other Signal\n");
    }
    printf("Sender PID: %d\n", info->si_pid);
    printf("Signal Code: %d\n", info->si_code);
    printf("Signal context captured successfully.\n");
    printf("-----\n");
}
int main(){
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_sigaction = signalHandler;
    sa.sa_flags = SA_SIGINFO;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);
    printf("Signal handlers registered.\n");
    printf("PID = %d\n", getpid());
    printf("\nTesting signal events...\n");
```

```
    printf("\nTesting signal events...\n");
    printf("1. Press Ctrl+C (SIGINT)\n");
    printf("2. Run: kill -SIGUSR1 %d\n", getpid());
    printf("3. Run: kill -SIGTERM %d\n", getpid());
    printf("Press Ctrl+Z to stop the program.\n\n");
    while (1)
    {
        printf("Program is running...\n");
        sleep(5);
    }
    return 0;
}
```

Output:



```
root@DESKTOP-VMN932K: ~  
root@DESKTOP-VMN932K:~# ./register  
Signal handlers registered.  
PID = 776  
  
Testing signal events...  
1. Press Ctrl+C (SIGINT)  
2. Run: kill -SIGUSR1 776  
3. Run: kill -SIGTERM 776  
Press Ctrl+Z to stop the program.  
  
Program is running...  
Program is running...  
Program is running...  
Signal Received: 10  
Handled SIGUSR1  
Sender PID: 704  
Signal Code: 0  
Signal context captured successfully.  
-----  
Program is running...  
Program is running...  
Signal Received: 15  
Handled SIGTERM  
Sender PID: 704  
Signal Code: 0  
Signal context captured successfully.  
-----  
Program is running...  
Program is running...  
Program is running...
```

```
root@DESKTOP-VMN932K:~# kill -SIGUSR1 776  
root@DESKTOP-VMN932K:~# kill -SIGTERM 776  
root@DESKTOP-VMN932K:~# ^C  
root@DESKTOP-VMN932K:~#
```

Task2:

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <signal.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
pid_t fg_pid = -1;  
void sigint_handler(int sig){  
    printf("\nParent received SIGINT (Ctrl+C)\n");  
    if (fg_pid > 0){  
        printf("Forwarding SIGINT to foreground process (PID %d)\n", fg_pid);  
        kill(fg_pid, SIGINT);  
    }  
    else{  
        printf("No foreground process running.\n");  
    }  
}  
int main(){  
    signal(SIGINT, sigint_handler);  
    fg_pid = fork();  
    if (fg_pid < 0){  
        perror("fork");  
        return 1;  
    }  
    if (fg_pid == 0){  
        signal(SIGINT, SIG_DFL);  
        printf("Foreground job started.\n");  
        printf("Child PID: %d\n", getpid());  
        printf("Press Ctrl+C to interrupt the child.\n");  
        while (1){  
            printf("Foreground job is running...\n");  
            sleep(2);  
        }  
    }  
    else{  
        int status;  
        printf("Shell PID: %d\n", getpid());  
        printf("Foreground PID: %d\n", fg_pid);  
        waitpid(fg_pid, &status, 0);  
        if (WIFSIGNALED(status)){
```

```

        if (WIFSIGNALED(status)){
            printf("\nForeground job terminated by signal %d\n",
                WTERMSIG(status));
        }
        else if (WIFEXITED(status)){
            printf("\nForeground job exited normally.\n");
        }
        fg_pid = -1;
        printf("Job state updated: Finished\n");
        printf("Shell is still running.\n");
    }
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./shell
Shell PID: 845
Foreground PID: 846
Foreground job started.
Child PID: 846
Press Ctrl+C to interrupt the child.
Foreground job is running...
^C
Parent received SIGINT (Ctrl+C)
Forwarding SIGINT to foreground process (PID 846)

Foreground job terminated by signal 2
Job state updated: Finished
Shell is still running.
root@DESKTOP-VMN932K:~#

```

Task3:

To Capture SIGTSTP, Suspend Processes, Update Job Table,
Preserve State, Resume Later, Test Suspension Workflow

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid;
int job_suspended = 0;
void sigtstp_handler(int sig){
    if (child_pid > 0){
        printf("\nShell: Captured SIGTSTP (Ctrl+Z)\n");
        kill(child_pid, SIGSTOP);
        job_suspended = 1;
        printf("Shell: Foreground job suspended.\n");
        printf("Shell: Job table updated (Stopped).\n");
        printf("Shell: State preserved.\n");
        printf("Use 'fg' simulation to resume.\n");
    }
}
int main(){
    signal(SIGTSTP, sigtstp_handler);
    child_pid = fork();
    if (child_pid < 0){
        perror("fork");
        return 1;
    }
    if (child_pid == 0){
        printf("Foreground Job Started\n");
        printf("Child PID: %d\n", getpid());
        while (1){
            printf("Child is running...\n");
            sleep(2);
        }
    }
    else{
        sleep(5);
        printf("\nPress Ctrl+Z to suspend the child process.\n");
        while (1){
            if (job_suspended){
                char choice;

```

```

                printf("\nResume suspended job? (y/n): ");
                scanf(" %c", &choice);
                if (choice == 'y' || choice == 'Y'){
                    printf("Shell: Resuming job...\n");
                    kill(child_pid, SIGCONT);
                    job_suspended = 0;
                    printf("Shell: Job table updated (Running).\n");
                }
                else{
                    printf("Shell: Terminating suspended job.\n");
                    kill(child_pid, SIGKILL);
                    wait(NULL);
                    printf("Job removed from table.\n");
                    break;
                }
            }
            sleep(1);
        }
    }
    return 0;
}

```

Output:

```
root@DESKTOP-VMN932K:~# ./test
Foreground Job Started
Child PID: 866
Child is running...
^ZChild is running...

Shell: Captured SIGTSTP (Ctrl+Z)
Shell: Foreground job suspended.
Shell: Job table updated (Stopped).
Shell: State preserved.
Use 'fg' simulation to resume.

Press Ctrl+Z to suspend the child process.

Resume suspended job? (y/n): y
Shell: Resuming job...
Shell: Job table updated (Running).
Child is running...
^ZChild is running...

Shell: Captured SIGTSTP (Ctrl+Z)
Shell: Foreground job suspended.
Shell: Job table updated (Stopped).
Shell: State preserved.
Use 'fg' simulation to resume.

Resume suspended job? (y/n): n
Shell: Terminating suspended job.
Job removed from table.
```

Task4:

To Create Process Groups, Assign Group IDs, Transfer
Terminal Control, Restore Ownership, Coordinate Signals,
Validate Behavior

```

#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <termios.h>
int main(){
    pid_t pid;
    int status;
    pid_t shell_pgid;
    signal(SIGTTOU, SIG_IGN);
    shell_pgid = getpgrp();
    printf("Shell PID : %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);
    pid = fork();
    if (pid < 0){
        perror("fork");
        exit(EXIT_FAILURE);
    }
    if (pid == 0){
        setpgid(0, 0);
        printf("\nChild Process Created\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PGID: %d\n", getpgrp());
        printf("Running for 10 seconds...\n");
        for (int i = 1; i <= 10; i++){
            printf("Child running... %d\n", i);
            sleep(1);
        }
        printf("Child exiting normally.\n");
        exit(0);
    }
    else{
        setpgid(pid, pid);
        sleep(1);
        printf("\nParent transferring terminal control to child...\n");
        if (tcsetpgrp(STDIN_FILENO, pid) == -1)
            perror("tcsetpgrp");
        waitpid(pid, &status, 0);
        printf("\nRestoring terminal control to parent...\n");

        printf("\nRestoring terminal control to parent...\n");
        if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
            perror("tcsetpgrp");
        printf("\nValidation:\n");
        printf(" Process group created.\n");
        printf(" Group ID assigned.\n");
        printf(" Terminal ownership transferred.\n");
        printf(" Terminal ownership restored.\n");
        printf(" Parent and child coordinated correctly.\n");
        if (WIFEXITED(status))
            printf("Child exited with status %d\n", WEXITSTATUS(status));
        printf("Program completed successfully.\n");
    }
    return 0;
}

```

Output:

```
root@DESKTOP-VMN932K:~# ./groups
Shell PID : 960
Shell PGID: 960

Child Process Created
Child PID : 961
Child PGID: 961
Running for 10 seconds...
Child running... 1

Parent transferring terminal control to child...
Child running... 2
Child running... 3
Child running... 4
Child running... 5
^C
Restoring terminal control to parent...

Validation:
  Process group created.
  Group ID assigned.
  Terminal ownership transferred.
  Terminal ownership restored.
  Parent and child coordinated correctly.
Program completed successfully.
```